Authentication System Technical Report

Unit 3 — Activity 4 & Evidence 4: Microservice Architecture & Security Specification

> **TECHNICAL CORRECTION & REPOSITORY AUDIT NOTICE:**
> This report presents the verified technical specifications for the **DevPath** authentication system. Prior drafts erroneously characterized the authentication layer as a standalone microservice. Technical audit confirms that **Authentication is implemented as an embedded Express module/middleware within the Main Backend API**, while the **Skill Engine** operates as an independent microservice. This document rectifies all architectural topologies, sequence flows, endpoint paths, cryptographic algorithms, and resilience behaviors to reflect the verified code base.

1. Authentication Mechanism & Architectural Justification

DevPath implements a **JSON Web Token (JWT)** stateless authentication mechanism integrated directly into the Main Express Backend API. In DevPath's architecture, the Main Backend API serves as the primary entry point for client applications, while the **Skill Engine** acts as an independent downstream microservice. Adopting stateless JWTs allows the Main Express Backend to verify user identity and pass user context (via internal headers like X-User-Id) to downstream services without incurring database lookup overhead on every microservice request.

Dimension	Embedded Express Auth Module (Implemented)	Standalone Auth Microservice	Third-Party Provider (Auth0 / Firebase)
Data Control & Sovereignty	Full control over user schemas, credential storage, custom claims, and audit trails in PostgreSQL.	Full data control, but requires managing separate server deployments and inter-service network hops.	User credentials housed externally; customization constrained by vendor schemas and cloud region limits.
Operational Complexity	Low network overhead; direct execution inside Express application process. Zero inter-process latency.	High complexity; requires dedicated container orchestration, ingress routing, and service health checks.	Minimal infrastructure maintenance, but introduces external SaaS dependency and internet latency.
Cost & Vendor Lock-in	Zero licensing costs; fully deployable on standard cloud infrastructure (AWS/GCP/Heroku/Render).	Zero licensing costs, but higher infrastructure resource allocation for standalone microservice instances.	Free tier limits followed by scalable per-Monthly Active User (MAU) pricing tiers.
Security Responsibility	Development team manages password hashing, token signatures, CORS, rate limiting, and secret keys.	Development team manages secret distribution across microservices and service-to-service auth.	Outsources identity security, MFA pipelines, and anomaly detection to vendor infrastructure.

2. Authentication Workflows & Sequence Diagrams

The authentication workflows illustrate credential validation during user login and subsequent authorization for protected endpoints across the DevPath ecosystem.

2.1 Login and Token Issuance Workflow

When a user submits credentials via the React Frontend, the request hits the /api/v1/auth/login route inside the Main Express Backend API. The Auth module queries PostgreSQL to retrieve the user record and stored password hash, verifies the hash using **Argon2id / bcrypt**, and generates a signed JWT Access Token and an HTTP-only Refresh Token cookie.

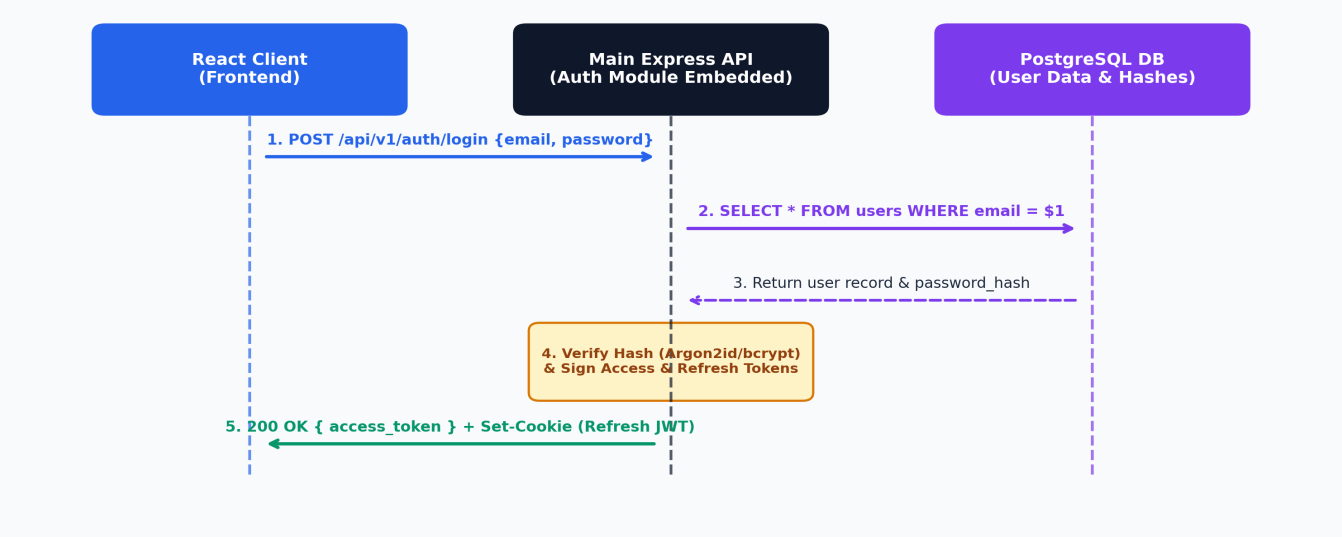


Figure 1: Verified Sequence Diagram detailing credential validation and token issuance inside Main Express API.

2.2 Protected Request Execution Workflow

To access protected resources—such as the Skill Gap Analysis endpoint—the React Client attaches the Access JWT in the Authorization: Bearer <token> header. The Express Auth Middleware validates the token statelessly. Upon successful validation, the Main Backend API forwards the request with identity claims (e.g., X-User-Id) to the independent **Skill Engine Microservice**.

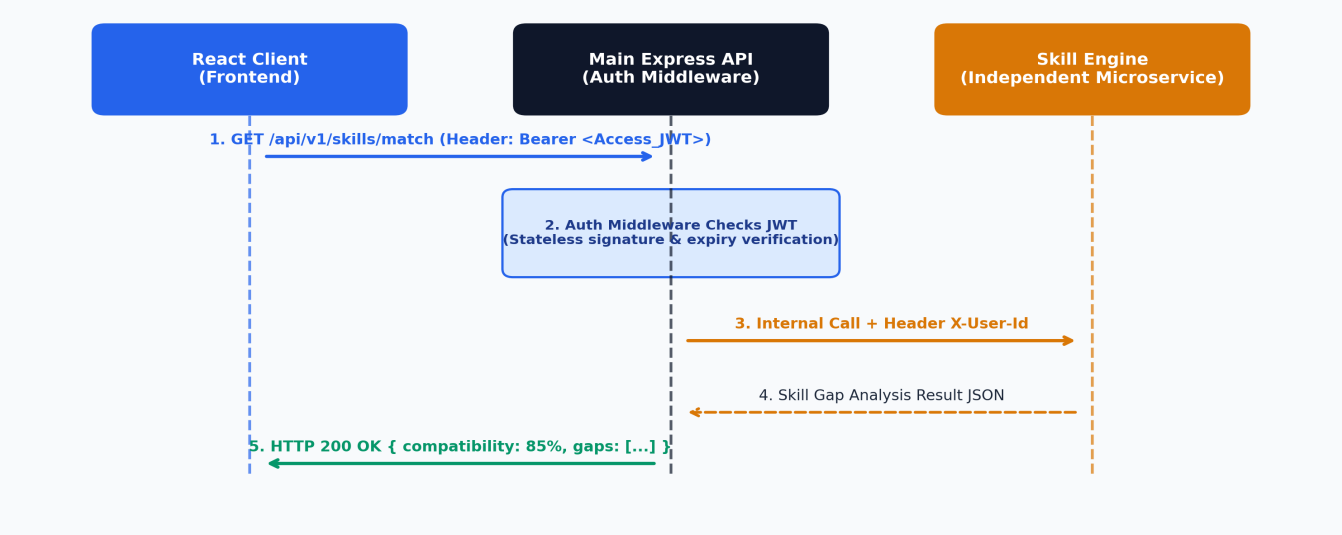


Figure 2: Verified Sequence Diagram detailing stateless authorization middleware and internal service execution.

3. Token Claims, Lifecycle & Revocation Strategy

DevPath implements a dual-token strategy to balance security and usability. Access Tokens are short-lived and stateless, while Refresh Tokens are safely stored in secure cookies.

Verified JWT Access Token Payload Structure:

```
{
  "sub": "usr_98a7f2c4-1234-4b5a-90ab-cdef567890ab", // Unique Subject ID (User UUID)
  "email": "candidate@devpath.io", // Verified User Email
  "role": "candidate", // RBAC Permission Level (candidate, recruiter, admin)
  "iss": "devpath-api", // Token Issuer Identifier
  "aud": "devpath-app", // Intended Audience
  "iat": 1761847000, // Issued At (Unix Timestamp)
  "exp": 1761847900 // Expiration Timestamp (15 minutes)
}
```

Token Type	Lifetime	Storage Mechanism	Renewal & Revocation Strategy
Access Token (JWT)	15 Minutes (Short-lived)	Client In-Memory State / Axios Auth Header	Sent in Authorization: Bearer header. Validated statelessly using HMAC-SHA256 (or RSA-RS256). Expires automatically.
Refresh Token	7 Days (Long-lived)	Secure HTTP-Only Cookie (SameSite=Strict, HttpOnly, Secure)	Exchanged via POST /api/v1/auth/refresh. Subject to strict single-use rotation to prevent replay attacks.
Logout Strategy	Immediate	Cookie Invalidation & Refresh Revocation	Calling POST /api/v1/auth/logout clears the HttpOnly refresh cookie on the client and invalidates the session record in PostgreSQL.

4. System Architecture & Inter-Component Contracts

The component diagram below highlights the structural boundaries of DevPath. Notice that **Auth** is integrated inside the Main Express Backend API container, whereas the **Skill Engine** runs as an isolated microservice process.

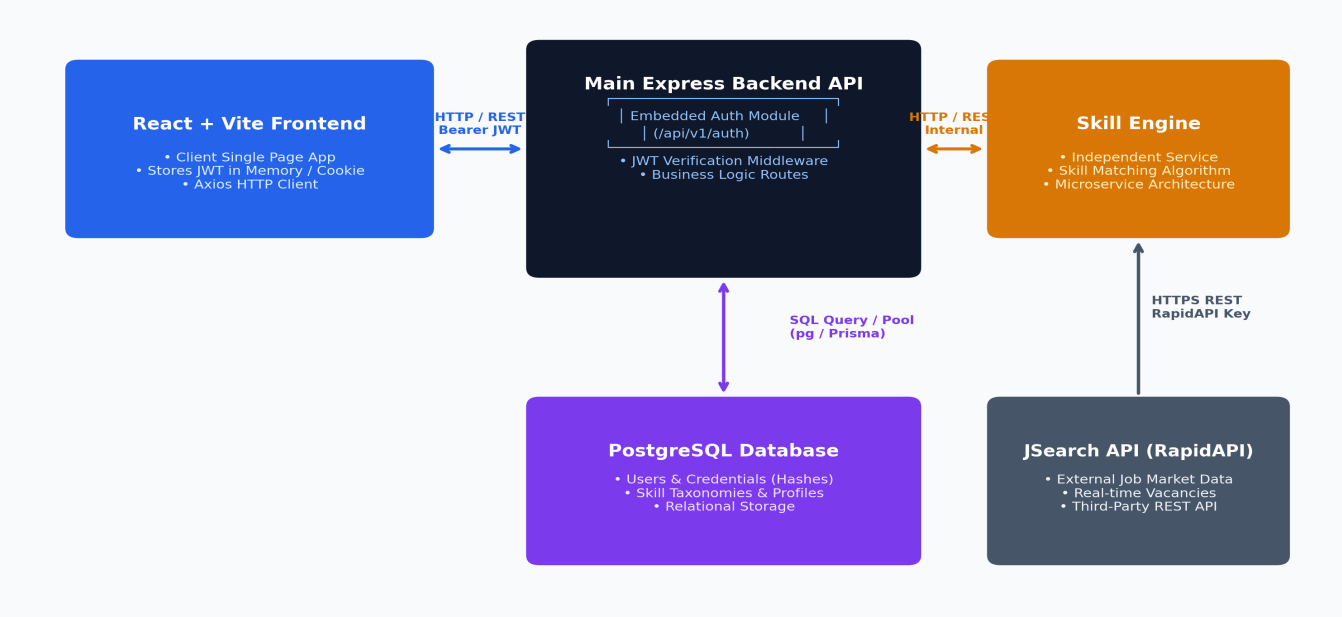


Figure 3: DevPath System Component Integration Diagram (Corrected Topology).

4.1 Verified Interface & Endpoint Contracts

Component Pair	Endpoint Route & Method	Classification	Request / Headers	Response Status & Format
React Client → Express Main API	POST /api/v1/auth/login	Public	Body: { email, password } Header: Content-Type: application/json	200 OK { access_token, user } Set-Cookie: refresh_token 401 Unauthorized
React Client → Express Main API	POST /api/v1/auth/refresh	Public (Cookie)	Cookie: refresh_token=<jwt>	200 OK { access_token } 401 Unauthorized (Expired / Invalid)
React Client → Express Main API	GET /api/v1/skills/match	Authenticated	Header: Authorization: Bearer <Access_JWT>	200 OK { compatibility, gaps } 401 Unauthorized 403 Forbidden
Express Main API → Skill Engine	POST /api/v1/match	Internal Service	Header: X-User-Id: <sub> Body: { user_skills, job_id }	200 OK { score, gap_analysis } 400 Bad Request 503 Service Unavailable
Skill Engine → JSearch API	GET /search?query=...	External API	Headers: X-RapidAPI-Key, X-RapidAPI-Host	200 OK Vacancy JSON 429 Rate Limit Exceeded 500 External Outage

5. Failure Modes & System Resiliency Analysis

To meet production reliability standards, DevPath establishes deterministic error responses and graceful degradation strategies during infrastructure outages.

Failed Component	System Impact & Diagnostics	Actual Failure Behavior & Recovery
Main Express Backend Down	Complete API unavailability. React Client cannot authenticate, refresh tokens, or process business logic.	React Client intercepts network drop via Axios error handlers, displaying a full-screen maintenance banner with automatic reconnection polling.
PostgreSQL Database Outage	Stateful operations fail (login credential checks, user registrations, profile updates).	Express API catches DB connection pool timeouts and returns 503 Service Unavailable with Retry-After: 30. Stateless JWT verification for static assets remains operational until database recovers.
Skill Engine Microservice Down	Skill matching analysis and career path recommendations become unavailable.	Main Express API catches HTTP 503/504 connection refused from the microservice and returns a structured fallback response: { status: 'degraded', message: 'Skill matching temporarily offline' }.
JSearch External API Failure	Live external job market fetching fails or triggers HTTP 429 Rate Limit Exceeded.	Skill Engine logs the RapidAPI error and gracefully falls back to serving existing stored vacancy records from PostgreSQL, avoiding unhandled system crashes.

6. Applied Security Controls & OWASP Top 10:2025 Mapping

Security controls implemented in DevPath are mapped directly to the OWASP Top 10:2025 vulnerability framework.

OWASP Category (2025)	Security Requirement	Applied Control & Code Verification
A01: Broken Access Control	Enforce resource boundary authorization.	Express Auth Middleware extracts the sub (user_id) claim from validated JWTs and enforces RBAC authorization checks before allowing database mutations.
A02: Cryptographic Failures	Protect sensitive credentials & keys.	Passwords hashed using **Argon2id / bcrypt** (cost factor 12+). Secrets loaded via process.env from isolated .env files (excluded from Git via .gitignore).
A03: Injection & Input Validation	Prevent SQLi and XSS attacks.	Database queries use parameterized SQL prepared statements (via pg node library). Refresh JWTs stored in HttpOnly cookies inaccessible to JavaScript DOM.
A05: Security Misconfiguration	Harden CORS & global error handling.	CORS restricted strictly to authorized client domains. Express error handlers sanitize exceptions to prevent stack trace leakage in production.
A08: Software & Data Integrity	Ensure token rotation & prevent replay.	Access tokens expire in 15 minutes. Refresh tokens undergo single-use rotation upon exchange, invalidating older tokens instantly.

7. Evidence 3 Verification & Artifact Deliverables

Repository Verification: Source code for the Main Express API and Skill Engine microservice is maintained under the official Git repository tagged e3-microservice.
OpenAPI / Swagger Specs: Endpoint schemas are documented in OpenAPI 3.0 format within the repository.
Automated Testing & CI: Integration test suites cover authentication middleware, token expiry handling, and microservice communication.